



Chapter 6

Comprehensions in Python

Writing elegant, efficient, and readable Python code

1. Introduction to Comprehensions
2. List Comprehension
3. Conditional List Comprehension
4. Nested List Comprehension
5. Dictionary Comprehension
6. Set Comprehension
7. Generator Expressions

Table of Contents

1. Introduction to Comprehensions	1
1.1 The Traditional Approach vs. Comprehensions	1
Traditional Approach — Using a Loop	1
Using a List Comprehension	1
1.2 Types of Comprehensions in Python	1
2. List Comprehension	1
2.1 Syntax of List Comprehension	1
2.2 Worked Examples — List Comprehension	1
Example A — Create a list of numbers from 1 to 10	1
Example B — Create a list of cubes	1
Example C — Convert temperatures from Celsius to Fahrenheit	1
Example D — Extract the first character from each word	1
Example E — Convert a list of strings to uppercase	1
Example F — Get the length of each word	1
Example G — Working with a string (iterating characters)	1
3. Conditional List Comprehension	1
3.1 Syntax — Filtering with a Condition	1
3.2 Worked Examples — Filtering	1
Example A — Extract only even numbers	1
Example B — Extract only positive numbers	1
Example C — Get names longer than 4 characters	1
Example D — Get words that start with a vowel	1
Example E — Extract numbers divisible by 3 or 5	1
3.3 If-Else in List Comprehension (Conditional Expression)	1
Example F — Label each number as 'even' or 'odd'	1
Example G — Replace negative numbers with 0	1
4. Nested List Comprehension	1
4.1 Syntax — Nested List Comprehension	1
4.2 Worked Examples — Nested Comprehension	1
Example A — Flatten a matrix (2D list to 1D list)	1
Example B — Generate all pairs (combinations)	1
Example C — Multiplication table using nested comprehension	1
Example D — Transpose a matrix	1
5. Dictionary Comprehension	1
5.1 Syntax — Dictionary Comprehension	1

5.2 Worked Examples — Dictionary Comprehension	1
Example A — Create a dictionary of squares	1
Example B — Map student names to their grade	1
Example C — Create a dictionary from a list of words with their lengths	1
Example D — Swap keys and values of a dictionary	1
Example E — Filter a dictionary based on a condition	1
Example F — Create a dictionary from a range with conditional values	1
6. Set Comprehension	1
6.1 Syntax — Set Comprehension	1
6.2 Worked Examples — Set Comprehension	1
Example A — Create a set of squares	1
Example B — Remove duplicates automatically	1
Example C — Get unique first characters from a list of words	1
Example D — Set comprehension with condition	1
Example E — Get unique lengths of words	1
7. Generator Expressions	1
7.1 What is a Generator?	1
7.2 Syntax — Generator Expression	1
7.3 Worked Examples — Generator Expressions	1
Example A — Creating a basic generator	1
Example B — Using next() to get values one at a time	1
Example C — Using a generator with sum()	1
Example D — Convert generator to a list	1
Example E — Generator with condition	1
7.4 Memory Comparison — List vs Generator	1

1. Introduction to Comprehensions

In programming, we often deal with collections of data — a list of numbers, a set of names, or a dictionary of records. A very common task is to take one collection, process each element somehow, and produce a new collection.

Python gives us a very powerful and clean feature to do this in a single line — called Comprehensions.

What is a Comprehension?

A comprehension is a concise, readable way to create a new collection (list, dictionary, or set) by applying an expression to each item in an existing iterable — all written in a single line of code.

1.1 The Traditional Approach vs. Comprehensions

To appreciate comprehensions, let us first look at how we solved the same problem before comprehensions were used.

Traditional Approach — Using a Loop

Suppose you have a list of numbers and you want to create a new list containing the square of each number.

Example 1.1 — Traditional method using a for loop

```
numbers = [1, 2, 3, 4, 5]
squares = []

for num in numbers:
    squares.append(num ** 2)

print(squares)
```

Output

```
[1, 4, 9, 16, 25]
```

This works correctly, but it takes 4 lines of code just to build a new list. Now let us look at how comprehension makes this simpler.

Using a List Comprehension

Example 1.2 — Same task using list comprehension

```
numbers = [1, 2, 3, 4, 5]
squares = [num ** 2 for num in numbers]

print(squares)
```

Output

```
[1, 4, 9, 16, 25]
```

Both examples produce exactly the same result. But the comprehension version is shorter, easier to read, and more Pythonic — meaning it follows the spirit of how Python code should be written.

Figure 1.1 — Comparing the traditional loop approach vs. comprehension

Aspect	Traditional Loop	Comprehension
Lines of code	4–5 lines	1 line
Readability	Standard	More concise
Performance	Normal	Often faster
Use in expressions	Cannot use inline	Can be used inline

1.2 Types of Comprehensions in Python

Python supports four types of comprehensions:

- List Comprehension — produces a list `[]`
- Dictionary Comprehension — produces a dictionary `{ key: value }`
- Set Comprehension — produces a set `{ }`
- Generator Expression — produces a generator `( )`

We will study each type in detail throughout this chapter.



Note for Students

All comprehensions follow the same basic structure. Once you understand list comprehension well, the others will feel very natural and easy to learn.

2. List Comprehension

A List Comprehension is a concise way to create a new list by applying an expression to each element of an existing iterable (such as a list, range, string, etc.).

Think of it as a compact version of a for loop that builds a list.

2.1 Syntax of List Comprehension

The general syntax of a list comprehension is:

```
new_list = [ expression for item in iterable ]
```

Figure 2.1 — Breaking down the syntax components

Part	What it means
new_list	The name of the new list being created
expression	The value or formula to apply to each item
for item in ...	Iteration — go through each element
iterable	Any collection you can loop over (list, range, string, etc.)

2.2 Worked Examples — List Comprehension

Example A — Create a list of numbers from 1 to 10

Example 2.1 — Using range() with list comprehension

```
numbers = [x for x in range(1, 11)]  
print(numbers)
```

Output

```
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

Here, the expression is simply x, so each element from range(1, 11) is added to the list as-is.

Example B — Create a list of cubes

Example 2.2 — Computing the cube of numbers 1 to 6

```
cubes = [x ** 3 for x in range(1, 7)]  
print(cubes)
```

Output

```
[1, 8, 27, 64, 125, 216]
```

Example C — Convert temperatures from Celsius to Fahrenheit

This is a very practical example. Suppose you have a list of temperatures in Celsius and you want to convert each one to Fahrenheit.

Example 2.3 — Temperature unit conversion

```
celsius = [0, 10, 20, 30, 40, 100]
fahrenheit = [(c * 9/5) + 32 for c in celsius]
print(fahrenheit)
```

Output

```
[32.0, 50.0, 68.0, 86.0, 104.0, 212.0]
```

Example D — Extract the first character from each word

Example 2.4 — String manipulation using list comprehension

```
subjects = ["Python", "Mathematics", "Science", "History"]
initials = [subject[0] for subject in subjects]
print(initials)
```

Output

```
['P', 'M', 'S', 'H']
```

Example E — Convert a list of strings to uppercase

Example 2.5 — Using string methods inside comprehension

```
names = ["alice", "bob", "charlie", "diana"]
upper_names = [name.upper() for name in names]
print(upper_names)
```

Output

```
['ALICE', 'BOB', 'CHARLIE', 'DIANA']
```

Example F — Get the length of each word

Example 2.6 — Applying len() inside a list comprehension

```
words = ["apple", "banana", "cherry", "date"]
lengths = [len(word) for word in words]
print(lengths)
```

Output

```
[5, 6, 6, 4]
```

Example G — Working with a string (iterating characters)

Strings are also iterable in Python — you can loop through each character.

Example 2.7 — Extracting vowels from a string using comprehension

```
sentence = "comprehension"
chars = [ch for ch in sentence]
print(chars)
```

Output

```
['c', 'o', 'm', 'p', 'r', 'e', 'h', 'e', 'n', 's', 'i', 'o', 'n']
```



Key Takeaway

In a list comprehension, the expression on the left can be any valid Python operation — arithmetic, string methods, function calls, and more. This makes it extremely flexible.

ETHANS
CLOUD & AI ACADEMY



3. Conditional List Comprehension

In many real-world scenarios, you do not want to include every element in your result — you want to filter certain elements based on a condition. Python allows you to add an if condition at the end of a list comprehension to control which elements are included.

3.1 Syntax — Filtering with a Condition

```
new_list = [ expression for item in iterable if condition ]
```

Figure 3.1 — Syntax with filtering condition

Part	What it means
expression	What to do with each item that passes the condition
for item in	Loop through each item in the iterable
if condition	Only include item if this condition is True

3.2 Worked Examples — Filtering

Example A — Extract only even numbers

Example 3.1 — Filter even numbers from a list

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
evens = [n for n in numbers if n % 2 == 0]
print(evens)
```

Output

```
[2, 4, 6, 8, 10]
```

The condition `n % 2 == 0` checks if a number is perfectly divisible by 2 (i.e., even). Only numbers passing this test are added to the list.

Example B — Extract only positive numbers

Example 3.2 — Filter positive numbers

```
data = [-5, -3, 0, 2, 7, -1, 9, 4]
positives = [x for x in data if x > 0]
print(positives)
```

Output

```
[2, 7, 9, 4]
```

Example C — Get names longer than 4 characters

Example 3.3 — Filter strings by length

```
names = ["Ali", "Priya", "Sam", "Nilufar", "Zoe"]
long_names = [name for name in names if len(name) > 4]
```

```
print(long_names)
```

Output

```
['Priya', 'Nilufar']
```

Example D — Get words that start with a vowel

Example 3.4 — Filter strings by starting character

```
words = ["apple", "banana", "orange", "grape", "avocado"]
vowel_words = [w for w in words if w[0] in "aeiou"]
print(vowel_words)
```

Output

```
['apple', 'orange', 'avocado']
```

Example E — Extract numbers divisible by 3 or 5

Example 3.5 — Multiple conditions using 'or'

```
numbers = list(range(1, 21))
result = [n for n in numbers if n % 3 == 0 or n % 5 == 0]
print(result)
```

Output

```
[3, 5, 6, 9, 10, 12, 15, 18, 20]
```

3.3 If-Else in List Comprehension (Conditional Expression)

You can also place an if-else directly inside the expression part (before the for keyword). This is used when you want to transform each element differently based on a condition — rather than just filtering.

```
new_list = [ value_if_true if condition else value_if_false for item in iterable ]
```

Example F — Label each number as 'even' or 'odd'

Example 3.6 — Using if-else inside expression

```
numbers = [1, 2, 3, 4, 5, 6]
labels = ["even" if n % 2 == 0 else "odd" for n in numbers]
print(labels)
```

Output

```
['odd', 'even', 'odd', 'even', 'odd', 'even']
```

Example G — Replace negative numbers with 0

Example 3.7 — Replacing values conditionally

```
values = [-3, 7, -1, 0, 4, -9, 2]
cleaned = [v if v >= 0 else 0 for v in values]
print(cleaned)
```

Output

```
[0, 7, 0, 0, 4, 0, 2]
```



Important Distinction

When using `if` at the END (after the `for` clause), it acts as a FILTER — only matching elements are included. When using `if-else` BEFORE the `for` clause, it acts as a TRANSFORMATION — every element is included but may be changed.

Figure 3.2 — Filtering vs. Transformation comparison

Type	Syntax Position	Purpose	All items included?
<code>if</code> (filter)	After <code>for</code> clause	Include only matching items	No
<code>if-else</code> (transform)	Before <code>for</code> clause	Transform each item	Yes

ETHANS
CLOUD & AI ACADEMY



4. Nested List Comprehension

So far, all our examples have dealt with a single list or iterable. But sometimes, you work with data that has multiple levels — for example, a matrix (a list of lists), or generating all combinations of two groups.

Nested list comprehension allows you to use more than one for clause within a single comprehension. It is the equivalent of writing a loop inside another loop — but in one line.

4.1 Syntax — Nested List Comprehension

```
result = [ expression for item1 in iter1 for item2 in iter2 ]
```

This is equivalent to the following nested loop structure:

Equivalent nested loop (for reference)

```
result = []
for item1 in iter1:
    for item2 in iter2:
        result.append(expression)
```

4.2 Worked Examples — Nested Comprehension

Example A — Flatten a matrix (2D list to 1D list)

A matrix is a list of lists. Flattening means converting it to a single flat list.

Example 4.1 — Flattening a 2D matrix

```
matrix = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]

flat = [num for row in matrix for num in row]
print(flat)
```

Output

```
[1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Reading the comprehension: for each row in matrix, go through each num in that row, and collect num.

Example B — Generate all pairs (combinations)

Suppose you want all possible pairs from two groups — like all combinations of colors and sizes.

Example 4.2 — All color-size combinations

```
colors = ["Red", "Blue", "Green"]
sizes = ["S", "M", "L"]

combinations = [(color, size) for color in colors for size in sizes]
```

```
for combo in combinations:
    print(combo)
```

Output

```
('Red', 'S')
('Red', 'M')
('Red', 'L')
('Blue', 'S')
('Blue', 'M')
('Blue', 'L')
('Green', 'S')
('Green', 'M')
('Green', 'L')
```

Example C — Multiplication table using nested comprehension**Example 4.3 — Generating a 3x3 multiplication table**

```
table = [[i * j for j in range(1, 4)] for i in range(1, 4)]
for row in table:
    print(row)
```

Output

```
[1, 2, 3]
[2, 4, 6]
[3, 6, 9]
```

Notice here the outer comprehension produces rows, and the inner one produces values for each row. The result is a list of lists — a matrix.

Example D — Transpose a matrix

Transposing a matrix means swapping rows with columns. Row 0 becomes column 0, row 1 becomes column 1, and so on.

Example 4.4 — Matrix transposition using nested comprehension

```
matrix = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]

transposed = [[row[i] for row in matrix] for i in range(3)]
for row in transposed:
    print(row)
```

Output

```
[1, 4, 7]
[2, 5, 8]
[3, 6, 9]
```



Reading Nested Comprehensions

When reading a nested list comprehension, always read the for clauses from left to right — the leftmost for is the outermost loop. The expression at the very beginning tells you what gets added to the result.



When to Avoid Nesting

Nested comprehensions are powerful, but deeply nested comprehensions (3+ levels) can become very hard to read and maintain. In those cases, using traditional nested for loops is often the better and cleaner choice.

ETHANS
CLOUD & AI ACADEMY



5. Dictionary Comprehension

Just as list comprehension builds a list, Dictionary Comprehension builds a dictionary. A dictionary stores data as key-value pairs, and dictionary comprehension lets you create one in a single, elegant line.

5.1 Syntax — Dictionary Comprehension

```
new_dict = { key_expr : value_expr for item in iterable }
```

Figure 5.1 — Dictionary comprehension syntax breakdown

Part	Description
{ }	Curly braces — produces a dictionary
key_expr	Expression that determines each key
value_expr	Expression that determines each value
for item in iterable	Loops over each element in the source

5.2 Worked Examples — Dictionary Comprehension

Example A — Create a dictionary of squares

Example 5.1 — Mapping numbers to their squares

```
squares = {x: x**2 for x in range(1, 6)}
print(squares)
```

Output

```
{1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

Here, each number x becomes the key, and x squared becomes the value.

Example B — Map student names to their grade

Example 5.2 — Creating a dictionary from two lists using zip()

```
students = ["Aarav", "Priya", "Zara", "Kiran"]
grades = [88, 92, 76, 95]

report = {name: grade for name, grade in zip(students, grades)}
print(report)
```

Output

```
{'Aarav': 88, 'Priya': 92, 'Zara': 76, 'Kiran': 95}
```

What is zip()?

zip() is a built-in Python function that pairs up elements from two lists. zip(['a','b'], [1, 2]) gives you ('a', 1) and ('b', 2). It is very commonly used with dictionary comprehension.

Example C — Create a dictionary from a list of words with their lengths

Example 5.3 — Word to length mapping

```
words = ["Python", "Java", "Kotlin", "Go"]
word_lengths = {word: len(word) for word in words}
print(word_lengths)
```

Output

```
{'Python': 6, 'Java': 4, 'Kotlin': 6, 'Go': 2}
```

Example D — Swap keys and values of a dictionary

Example 5.4 — Inverting a dictionary

```
original = {"a": 1, "b": 2, "c": 3}
inverted = {v: k for k, v in original.items()}
print(inverted)
```

Output

```
{1: 'a', 2: 'b', 3: 'c'}
```

Example E — Filter a dictionary based on a condition

You can also add an if condition to filter which items are included in the new dictionary.

Example 5.5 — Keep only students who passed (marks >= 40)

```
marks = {"Rohan": 72, "Sneha": 38, "Arjun": 55, "Tara": 29, "Dev": 91}
passed = {name: m for name, m in marks.items() if m >= 40}
print(passed)
```

Output

```
{'Rohan': 72, 'Arjun': 55, 'Dev': 91}
```

Example F — Create a dictionary from a range with conditional values

Example 5.6 — Classify numbers as even or odd

```
classify = {n: "even" if n % 2 == 0 else "odd" for n in range(1, 8)}
print(classify)
```

Output

```
{1: 'odd', 2: 'even', 3: 'odd', 4: 'even', 5: 'odd', 6: 'even', 7: 'odd'}
```


ETHANS
CLOUD & AI ACADEMY



6. Set Comprehension

Set Comprehension works exactly like list comprehension, but instead of producing a list, it produces a set. Recall that a set in Python is an unordered collection that automatically removes duplicate values.

This makes set comprehension particularly useful when you want unique results from a collection.

6.1 Syntax — Set Comprehension

```
new_set = { expression for item in iterable }
```

Key Difference

Set comprehension uses curly braces { } just like dictionary comprehension, but does NOT have a colon (:) separating key and value — it only has a single expression, so Python knows it is a set.

6.2 Worked Examples — Set Comprehension

Example A — Create a set of squares

Example 6.1 — Set of squares from 1 to 5

```
squares_set = {x**2 for x in range(1, 6)}  
print(squares_set)
```

Output

```
{1, 4, 9, 16, 25}
```

Note: The output order may vary — sets in Python are unordered.

Example B — Remove duplicates automatically

This is where set comprehension truly shines. Suppose you have a list with duplicates and you want only the unique values.

Example 6.2 — Getting unique values from a list with duplicates

```
scores = [85, 90, 85, 70, 90, 95, 70, 85]  
unique_scores = {s for s in scores}  
print(unique_scores)
```

Output

```
{70, 85, 90, 95}
```

Example C — Get unique first characters from a list of words

Example 6.3 — Unique starting letters

```
words = ["apple", "avocado", "banana", "blueberry", "cherry", "apricot"]  
first_letters = {w[0] for w in words}  
print(first_letters)
```

Output

{'a', 'b', 'c'}

Example D — Set comprehension with condition

Example 6.4 — Unique even numbers from a list

numbers = [1, 2, 2, 3, 4, 4, 5, 6, 6, 7, 8]
unique_evens = {n for n in numbers if n % 2 == 0}
print(unique_evens)

Output

{2, 4, 6, 8}

Example E — Get unique lengths of words

Example 6.5 — Unique word lengths

words = ["cat", "bat", "hat", "elephant", "ant", "bee", "ox"]
unique_lengths = {len(w) for w in words}
print(unique_lengths)

Output

{2, 3, 8}

Figure 6.1 — Comparison: List vs Set comprehension

Feature	List Comprehension []	Set Comprehension { }
Output type	list	set
Preserves order	Yes	No
Allows duplicates	Yes	No (auto-removed)
Syntax	[expr for x in it]	{ expr for x in it }

7. Generator Expressions

A Generator Expression looks almost identical to a list comprehension — the only difference is that it uses parentheses () instead of square brackets []. However, the behavior is fundamentally different and more memory-efficient.

7.1 What is a Generator?

When you write a list comprehension, Python immediately creates the entire list and stores it in memory. For small lists, this is fine.

But imagine you need to process one million numbers. Creating a list of one million items uses a large amount of memory — even if you only ever need one number at a time.



Generator Concept

A generator does NOT create all values at once. Instead, it computes and yields one value at a time — only when you ask for it. This is called lazy evaluation. It is extremely memory-efficient for large datasets.

7.2 Syntax — Generator Expression

```
gen_expr = ( expression for item in iterable )
```

7.3 Worked Examples — Generator Expressions

Example A — Creating a basic generator

Example 7.1 — Creating and consuming a generator

```
gen = (x**2 for x in range(1, 6))
print(gen)    # Does not print values — just shows generator object

for val in gen:
    print(val)
```

Output

```
<generator object <genexpr> at 0x7f1234abcd00>
1
4
9
16
25
```

Notice that printing `gen` directly does not show the values — it just shows that it is a generator object. The values are only produced when you iterate over it.

Example B — Using `next()` to get values one at a time

Example 7.2 — Manually stepping through a generator with next()

```
gen = (x * 10 for x in range(1, 5))

print(next(gen)) # Get 1st value
print(next(gen)) # Get 2nd value
print(next(gen)) # Get 3rd value
```

Output

```
10
20
30
```

Each call to `next()` computes and returns just the next value. The others are not computed yet.

Example C — Using a generator with sum()

Generator expressions work directly inside functions like `sum()`, `max()`, `min()`, and `list()`. This is where they are extremely practical.

Example 7.3 — Summing squares using a generator expression

```
total = sum(x**2 for x in range(1, 101))
print(total)
```

Output

```
338350
```

Here, the generator is passed directly to `sum()` without even storing it in a variable. Python computes each square one at a time and accumulates the total — never storing all 100 values at once.

Example D — Convert generator to a list**Example 7.4 — Materializing a generator into a list**

```
gen = (n * 3 for n in range(1, 6))
result = list(gen)
print(result)
```

Output

```
[3, 6, 9, 12, 15]
```

Example E — Generator with condition**Example 7.5 — Generator that yields only odd squares**

```
odd_squares = (x**2 for x in range(1, 11) if x % 2 != 0)
for val in odd_squares:
    print(val)
```

Output

```
1
9
25
49
81
```

7.4 Memory Comparison — List vs Generator

Example 7.6 — Comparing memory usage (using `sys.getsizeof`)

```
import sys

# List comprehension — stores everything in memory
squares_list = [x**2 for x in range(1000)]

# Generator expression — stores only a tiny generator object
squares_gen = (x**2 for x in range(1000))

print("List size :", sys.getsizeof(squares_list), "bytes")
print("Generator :", sys.getsizeof(squares_gen), "bytes")
```

Output

```
List size : 8856 bytes
Generator : 208 bytes
```

The list holds all 1000 values (over 8,000 bytes), while the generator holds just an object with instructions for how to produce values (only 208 bytes). As the range grows to millions, this difference becomes enormous.

Figure 7.1 — List Comprehension vs Generator Expression

Aspect	List Comprehension []	Generator Expression ()
Syntax	[expr for x in it]	(expr for x in it)
Returns	list object	generator object
Memory usage	High — stores all values	Low — one value at a time
Reusable?	Yes — iterate multiple times	No — exhausted after one use
Best for	Small-medium data, reuse	Large data, single-pass use

⚠ Generators are Exhausted After One Use

Once a generator has been fully consumed (iterated through), it cannot be restarted or reused. If you need to iterate over the data again, you must create a new generator.

Chapter Summary

This chapter introduced Python Comprehensions — one of the most powerful and distinctive features of the language. Below is a summary of everything covered.

Comprehension Type	Syntax	Returns	Key Use Case
List Comprehension	[expr for x in it]	list	Transform or filter a sequence
Conditional (filter)	[expr for x in it if cond]	list	Include only matching elements
Conditional (transform)	[a if c else b for x in it]	list	Apply different logic per item
Nested List	[expr for x in it for y in it2]	list	Process 2D data, combinations
Dictionary Comprehension	{ k: v for x in it }	dict	Build mappings from sequences
Set Comprehension	{ expr for x in it }	set	Unique values from a sequence
Generator Expression	(expr for x in it)	generator	Memory-efficient single-pass iteration



Core Principles to Remember

1. All comprehensions follow the same pattern: expression + for clause + optional condition.
2. Use List Comprehension for general-purpose collection building.
3. Use Dictionary Comprehension when building key-value pairs.
4. Use Set Comprehension when unique values are needed.
5. Use Generator Expressions when working with large datasets to save memory.

With a solid understanding of Python Comprehensions, you are now equipped to write cleaner, faster, and more expressive Python code — a fundamental skill for any Python programmer.